

VitalX – Decentralized Health Record Management System

SAVITRIBAI PHULE PUNE UNIVERSITY



A Project Report On

Sharayu Deogoankar (105)

Asha Kumbhar (09)

Sarthak Dhere (29)

Nishit Bhalerao (18)

Rajas Badade (12)

Under The Guidance of

Assistant Prof. Sahil Shaikh



Accepted By

Department of Technology

Savitribai Phule Pune University, Pune

Contents :-

2 PROJECT OVERVIE	5
2.1 Problem Statement	5
2.2 Objectives	5
2.3 Project Summary	6
3 TECHNICAL KEYWORDS	7
3.1 Area of Project	7
3.2 Technical Keywords	7
4 INTRODUCTION	9
4.1 Project Idea	9
4.2 Motivation	9
5 PROBLEM DEFINITION AND SCOPE	10
5.1 Problem Statement	10
5.2 Goals and Objectives	10
5.3 Statement of Scope	11
5.4 Major Constraints	11
5.5 Hardware Requirements	12
5.6 Software Requirements	12
6 SOFTWARE REQUIREMENT SPECIFICATION (SRS)	13
6.1 Introduction	13
6.2 Purpose & Scope of SRS	13
6.3 Responsibilities of Developer	14
6.4 Usage Scenario	14

6.5 User Profiles	14
6.6 Use Cases	15
6.7 Use Case Diagram	15
6.8 Data Description	16
6.9 ER Diagram	16
6.10 Data Flow Diagram	17
6.11 Activity Diagram	17
6.12 Non-Functional Requirements	18
6.13 State Diagram	18
6.14 Design Constraints	18
6.15 Software Interface Description	19
7 DESIGN DOCUMENT	20
7.1 System Architecture Diagram	20
7.1.1 System Architecture Diagram (Textual Representation)	20
7.2 Class Diagram	21
7.3 Additional Interface Diagrams	22
7.3.1 Wallet Authentication Interface	22
7.3.2 File Upload Interface	22
7.3.3 Record Viewing Interface	23
8 PROJECT IMPLEMENTATION	24
8.1 Smart Contract Implementation	24
8.2 Frontend Overview	24
8.3 Backend Overview	25
8.4 Architecture Flow	26
8.5 Architecture Flow Diagram (Textual Representation)	26
9 TESTING PAGES	27
9.1 System Testing	27

9.2 Types of Testing Performed	27
9.2.1 Unit Testing	27
9.2.2 Integration Testing	28
9.2.3 Functional Testing	29
9.2.4 Security Testing	29
9.2.5 Performance Testing	29
9.2.6 User Acceptance Testing	29
9.2.7 Test Case Tables	29
10 SCREENSHOTS OF PROJECT	30
10.1 Deployment	30
10.2 Metamask Connection	31
10.3 UI Screens	32
10.4 User Flow	33
10.5 Contract Interactions	34
11 CODE SNIPPETS PAGES	35
12 PROJECT TIMELINE	37
13 CHALLENGES AND SOLUTIONS	39
14 RISKS & MITIGATION	41
15 GLOSSARY	45
16 CONCLUSION	47
17 REFERENCES	49

CHAPTER 2 :- PROJECT OVERVIEW

2.1 Problem Statement :-

Traditional health record management systems are primarily centralized and controlled by hospitals or third-party service providers. These systems suffer from several limitations such as lack of transparency, data manipulation risks, unauthorized access, data breaches, and loss of patient ownership over medical information.

Patients have no direct control over who accesses their medical records, and health data is often scattered across multiple institutions, leading to inefficiency and repeated medical records.

Hence, there is a strong need for a secure, decentralized, and patient-centric health record management system that ensures data integrity, privacy, transparency, and complete ownership of medical records by patients.

2.2 Objectives :-

The main objectives of the VitalX – Decentralized Health Record Management System are:-

1. To design a decentralized platform for secure storage and management of health records.
2. To ensure patient ownership and control over medical data using blockchain-based identities.
3. To store medical records securely using IPFS and maintain immutable metadata on the blockchain.
4. To prevent unauthorized access, data tampering, and record manipulation.
5. To provide a user-friendly web interface using React for uploading and viewing health records.
6. To improve transparency, trust, and efficiency in healthcare data management systems.

2.3 Project Summary :-

The scope of the VitalX project includes the development of a decentralized health record management system that allows patients to securely upload, store, and access their medical records. The system uses Web3 wallet authentication (Privy) for secure user identification, IPFS for decentralized file storage, and blockchain smart contracts for storing immutable record metadata such as file hash, timestamp, and owner address.

VitalX provides functionalities such as:

- Secure wallet-based login
- Uploading and managing medical records
- Immutable logging of health data transactions
- Viewing medical history through a patient dashboard

The project focuses on patient-centric data control and aims to reduce dependency on centralized healthcare databases, thereby enhancing data security, privacy, and trust.

CHAPTER 3:-TECHNICAL KEYWORDS

3.1 Area of Project :-

The project VitalX – Decentralized Health Record Management System falls under the following areas:

1. **Healthcare Information Systems:** Digital management and secure handling of patient medical records.
2. **Blockchain Technology:** Use of distributed ledger technology to ensure immutability, transparency, and trust.
3. **Decentralized Applications: (DApps)** A Web3-based application that eliminates centralized control over sensitive health data.
4. **Data Security and Privacy:** Ensuring confidentiality and integrity of medical records using cryptography and wallets.
5. **Decentralized Storage Systems:** Storage of medical documents using IPFS instead of centralized servers.
6. **Web3 Authentication Systems:** Secure user authentication using wallet-based identity management (Privy).

3.2 Technical Keywords:-

1. **Blockchain** – A decentralized and immutable digital ledger used to store transaction data securely.
2. **Smart Contracts** – Self-executing programs deployed on blockchain that enforce rules automatically.
3. **Decentralization** – Distribution of control and data across a network rather than a single authority.
4. **IPFS (Inter Planetary File System)** – A peer-to-peer distributed file storage system.
5. **Web3** – The decentralized version of the web using blockchain and wallets.
6. **Cryptographic Hash** – A unique digital fingerprint used to verify file integrity.
7. **Ethereum** – A blockchain platform used for deploying smart contracts.

- 8. Wallet Authentication – Secure login using cryptographic wallet addresses.**
- 9. Privy Authentication – A Web3 authentication solution used for wallet-based login.**
- 10. React.js – A JavaScript library used for building the frontend user interface.**
- 11. Ethers.js – A JavaScript library used to interact with Ethereum blockchain.**
- 12. Immutability – The property of blockchain data that prevents modification after storage.**

CHAPTER 4:- INTRODUCTION PAGE

4.1 Project Idea :-

The project VitalX – Decentralized Health Record Management System aims to provide a secure, transparent, and patient-centric solution for managing medical records using blockchain technology. In traditional healthcare systems, medical data is stored in centralized databases controlled by hospitals or institutions, which makes the data vulnerable to unauthorized access, manipulation, and loss.

VitalX introduces a decentralized approach where patients have complete ownership of their medical records. The system uses blockchain to store immutable metadata of medical records and IPFS (Inter Planetary File System) to store the actual medical files securely. Smart contracts are used to manage record creation and , ensuring that once data is stored, it cannot be altered.

The frontend of the system is developed using React.js, providing a user-friendly interface, while wallet-based authentication (Privy) ensures secure and decentralized identity management. This architecture eliminates intermediaries and provides trust, transparency, and security in healthcare data management.

4.2 Motivation :-

The motivation behind developing VitalX arises from the growing challenges in existing healthcare record management systems. Current systems are mostly centralized, fragmented, and lack transparency, leading to problems such as data breaches, unauthorized data sharing, duplication of medical tests, and loss of patient trust.

Patients do not have full control over their own medical data, and transferring records between hospitals is time-consuming and inefficient. Additionally, centralized databases are prone to cyber-attacks and system failures.

With the advancement of blockchain technology, it is now possible to build a system that ensures data integrity, privacy, and patient ownership.

CHAPTER 5 :-PROBLEM DEFINITION & SCOPE

5.1 Problem Statement :-

The existing healthcare record management systems are primarily centralized and fragmented, where patient medical data is stored and controlled by hospitals or third-party institutions. Such systems are vulnerable to data breaches, unauthorized access, data manipulation, and single-point failures. Patients have limited control over their own medical records, and sharing records between healthcare providers is often slow, inefficient, and insecure.

Moreover, centralized databases lack transparency and auditability, making it difficult to verify the authenticity of medical records. These issues result in loss of trust, duplication of medical tests, increased healthcare costs, and compromised patient privacy. Therefore, there is a strong need for a secure, decentralized, and tamper-proof health record management system that ensures patient ownership, data integrity, and transparency.

5.2 Goals & Objectives :-

The main goals and objectives of the VitalX project are:

1. To design a decentralized health record management system using blockchain technology.
2. To ensure patient ownership and control over medical records.
3. To store medical records securely using IPFS and maintain immutable metadata on blockchain.
4. To provide a secure and user-friendly interface using React.js.
5. To enhance data privacy, integrity, and transparency in healthcare systems.

5.3 Statement of Scope :-

The scope of the VitalX project includes:

1. Development of a web-based decentralized application for managing health records.
2. Wallet-based authentication using Privy for secure user identity.
3. Uploading and storing medical records using IPFS.
4. Logging of medical record metadata on blockchain using smart contracts.
5. Viewing uploaded records through a patient dashboard.
6. Providing a scalable architecture that can be extended to include doctors and hospitals in the future.

5.4 Major Constraints :-

The major constraints associated with the project are:

1. **Regulatory Constraints:** Compliance with healthcare data protection laws and privacy regulations.
2. **Blockchain Transaction Cost:** Gas fees required for storing metadata on blockchain.
3. **Scalability Limitations:** Performance may depend on blockchain network congestion.
4. **Technical Knowledge:** Users must have basic understanding of wallet-based authentication.
5. **Network Dependency:** Requires stable internet connectivity and access to blockchain networks.
6. **IPFS Availability:** Availability of IPFS nodes and pinning services.

5.5 Hardware Requirements :-

Hardware Resource	Minimum Requirement	Purpose
Processor (CPU)	2.5 GHz or higher	Smart contract compilation and frontend processing
RAM	8 GB or higher	Running React app and blockchain tools
Storage	256 GB SSD	Project files and dependencies
Internet Connection	10 Mbps or higher	Blockchain and IPFS connectivity
System Type	64-bit System	Compatibility with development tools

Table. 1

5.6 Software Requirements :-

Software	Version	Purpose
Operating System	Windows / Linux / macOS	Development environment
Node.js	Latest LTS	Running React and backend services
React.js	18 or higher	Frontend development
Solidity	0.8.x or higher	Smart contract development
Hardhat / Truffle	Latest	Smart contract deployment and testing
IPFS / Pinata	Latest	Decentralized file storage
Privy Authentication	Latest	Wallet-based user authentication
Ethers.js	5.x or higher	Blockchain interaction
Visual Studio Code	Latest	Code editor

Table. 2

CHAPTER 6:- SOFTWARE REQUIREMENT SPECIFICATION

6.1 Introduction :-

The VitalX – Decentralized Health Record Management System is a blockchain-based application designed to securely manage and store medical records in a decentralized manner. The system leverages blockchain technology, smart contracts, and IPFS to ensure data immutability, transparency, and patient ownership.

The frontend of the system is developed using React.js, and secure user authentication is achieved through wallet-based authentication (Privy).

6.2 Purpose & Scope of SRS :-

Purpose :-

The purpose of this Software Requirement Specification (SRS) document is to provide a detailed description of the functional and non-functional requirements of the VitalX system. It serves as a reference for developers, testers, and evaluators to understand the system's expected behavior and constraints.

Scope :-

The scope of the SRS includes:

- Secure patient login using wallet authentication
- Uploading medical records to IPFS
- Storing record metadata on blockchain
- Viewing uploaded records
- Ensuring secure and immutable data handling

6.3 Responsibilities of Developer :-

The responsibilities of the developer include :-

- 1. Designing and implementing the React-based frontend.**
- 2. Integrating Privy wallet authentication for secure login.**
- 3. Developing and deploying smart contracts using Solidity.**
- 4. Integrating IPFS for decentralized file storage.**
- 5. Ensuring secure interaction between frontend and blockchain.**
- 6. Performing system testing and debugging.**
- 7. Preparing proper documentation for the project.**

6.4 Usage Scenario :-

A patient accesses the VitalX application and logs in using wallet-based authentication. After successful authentication, the patient uploads a medical record such as a prescription or lab report. The file is stored on IPFS, and the returned IPFS hash along with metadata is stored on the blockchain through a smart contract.

Later, the patient can view all uploaded records through the dashboard, ensuring transparency and data integrity.

6.5 User Profiles :-

The VitalX system currently supports a single user module, i.e., the Patient module. The patient is the sole authorized user who can upload, store, and view medical records. No doctor, hospital, or third-party modules are included in the current system implementation.

Patient Module Description:-

The patient logs into the system using wallet-based authentication and has complete ownership and control over their medical records. The patient can upload medical files, store them securely using IPFS, and view previously uploaded records through the dashboard. All actions are securely recorded on the blockchain to ensure immutability and transparency.

6.6 Use Cases :-

Use Case	Actor	Description
User Login	Patient	Login using wallet authentication
Upload Record	Patient	Upload medical record to IPFS
Upload Record	Patient	record to IPFS
Store Metadata	System	Store IPFS hash on blockchain
	Patient	View uploated medical records
View Records	Patient	
Verify Record	System	Verify record authenticity

Table. 3

6.7 Use Case Diagram :-

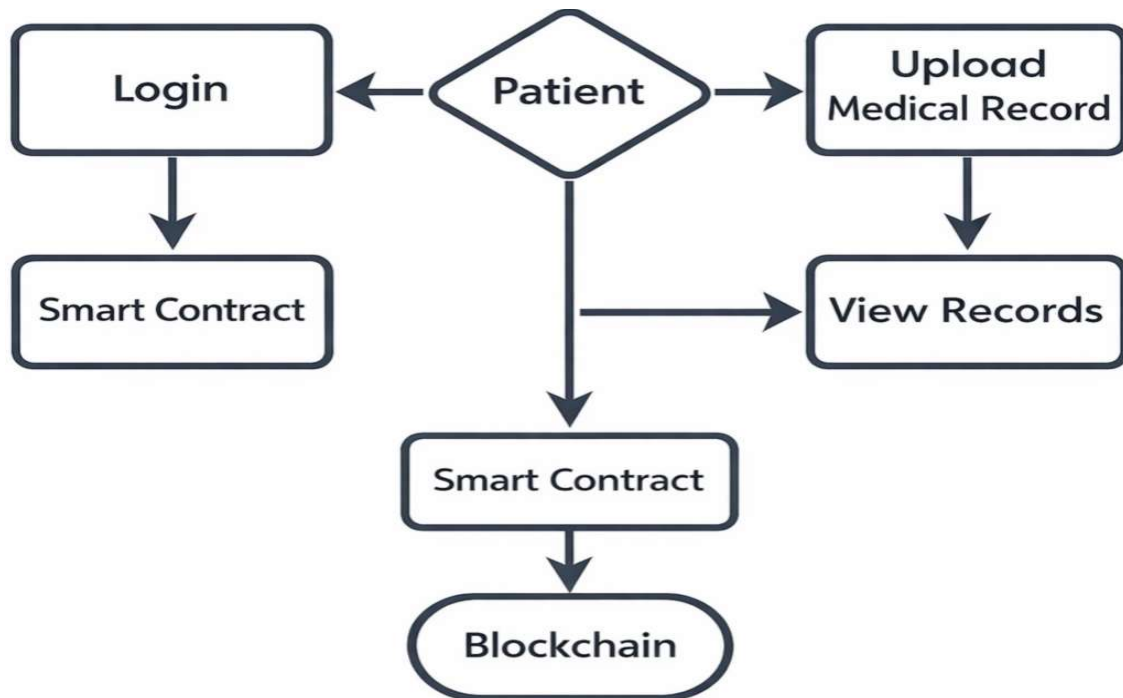


Figure 1

6.8 :- Data Description :-

Use Case	Actor	Description
User Login	Patient	Login using wallet authentication
Upload Record	Patient	Upload medical record to IPFS
Store Metadata	System	Store IPFS hash on blockchain
View Records	Patient	View uploaded medical records
Verify Record	System	Verify record authenticity

Table. 4

6.9 ER Diagram :-

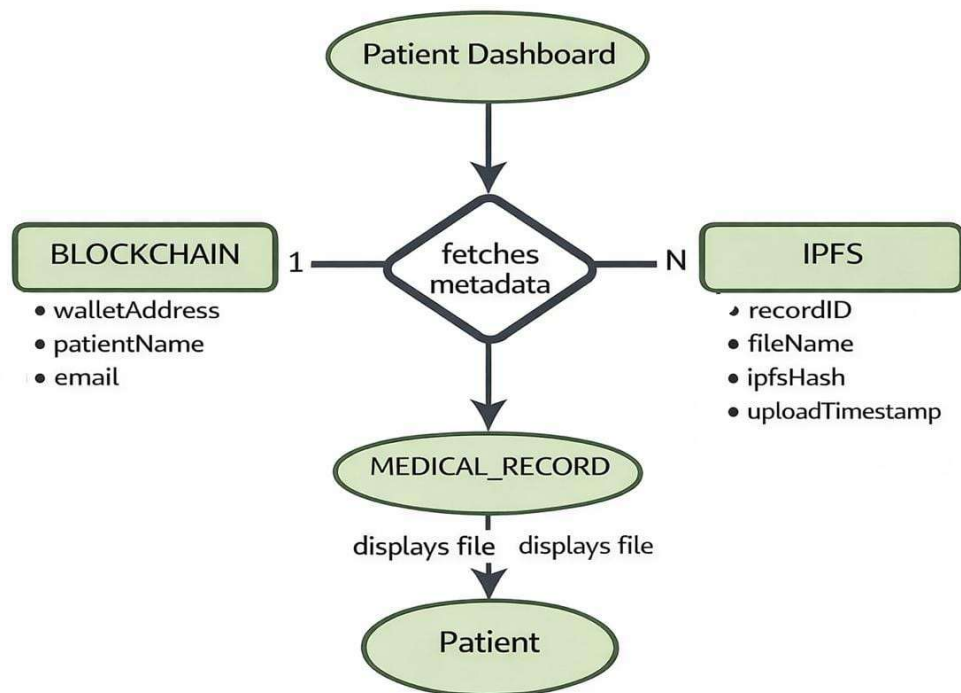


Figure. 2

6.10 Data Flow Diagram :-

Level 0 DFD :-



Level 1 DFD :-



Figure. 3

6.11 Activity Diagram :-

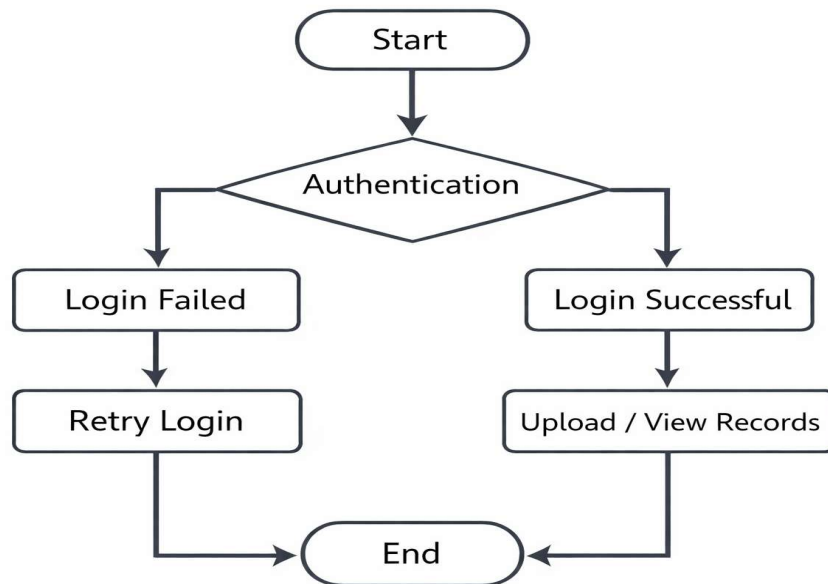


Figure. 4

6.12 Non-Functional Requirements :-

1. **Security:** Wallet-based authentication and encrypted storage.
2. **Scalability:** Support multiple users and records.
3. **Reliability:** Immutable blockchain storage ensures data safety.
4. **Performance:** Efficient IPFS retrieval and blockchain queries.
5. **Usability:** Simple and intuitive user interface.
6. **Availability:** System accessible anytime with internet access.

6.13 State Diagram :-



Figure. 5

6.14 Design Constraints :-

1. Medical files must not be stored directly on blockchain.
2. Only authenticated users can access the system.
3. IPFS hash must be valid before blockchain storage.
4. System must operate within blockchain gas limits.
5. Compliance with healthcare data privacy laws.

6.15 Software Interface Description :-

The system interfaces include:

- 1. User Interface: React.js web application.**
- 2. Authentication Interface: Privy wallet authentication system.**
- 3. Blockchain Interface: Ethers.js communication with smart contracts.**
- 4. Storage Interface: IPFS API for decentralized file storage.**

CHAPTER 7 :- DESIGN DOCUMENT

7.1 System Architecture Diagram :-

The system architecture of VitalX – Decentralized Health Record Management System follows a decentralized client–blockchain–storage model. The architecture ensures secure authentication, decentralized storage of medical records, and immutable logging of record metadata.

7.1.1 System Architecture Diagram (Textual Representation)

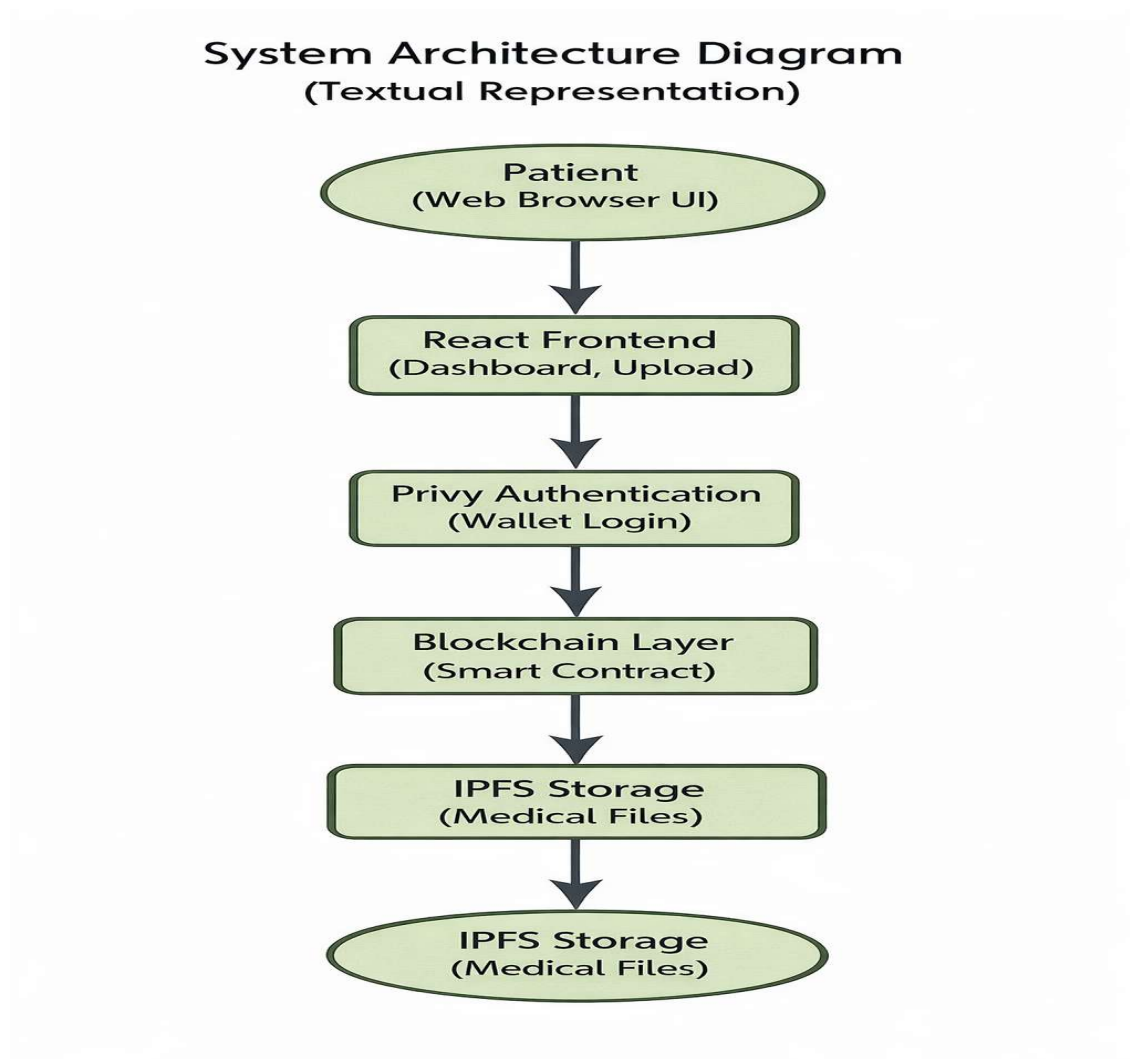


Figure. 6

Architecture Explanation

- The Patient accesses the system through a web browser.
- The React Frontend provides login, upload, and record viewing functionalities.
- Privy Authentication ensures secure wallet-based user authentication.
- The Blockchain Layer stores immutable metadata (IPFS hash, timestamp).
- IPFS stores the actual medical records securely in a decentralized manner.

7.2 Class Diagram :-

The class diagram represents the internal structure of the system and the interaction between different components.

Class Diagram (Textual Representation)

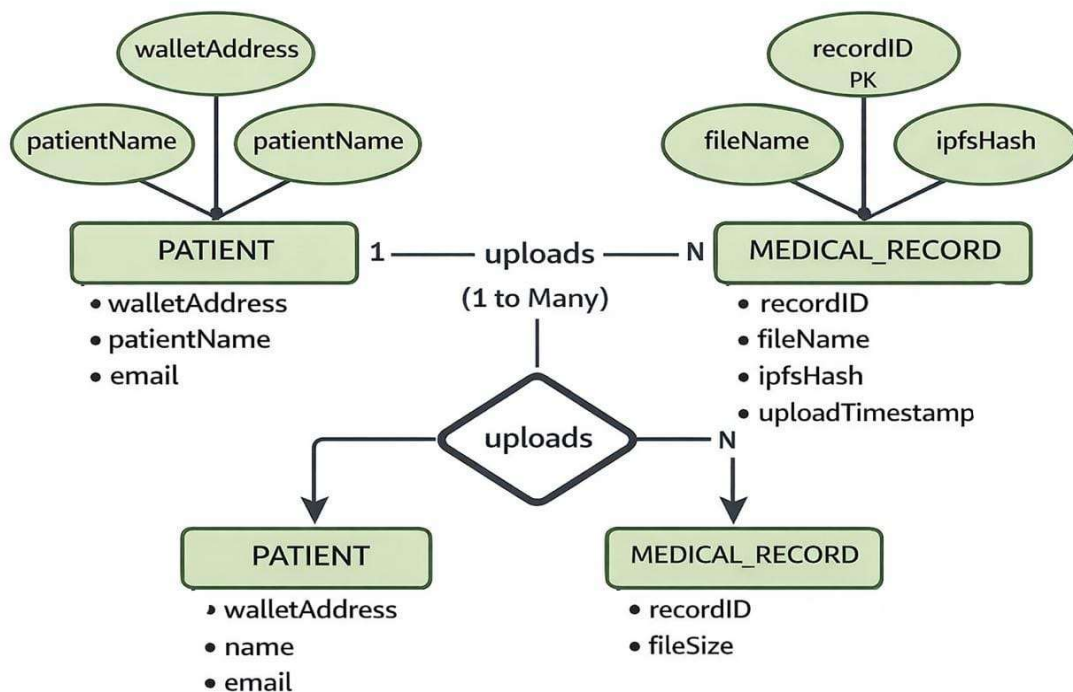


Figure. 7

Class Diagram Explanation :-

- The Patient class represents the authenticated user.
- The MedicalRecord class stores metadata of uploaded medical records.
- The SmartContract class handles blockchain interactions.
- A one-to-many relationship exists between Patient and MedicalRecord.

7.3 Additional Interface Diagrams :-

7.3.1 Wallet Authentication Interface :-

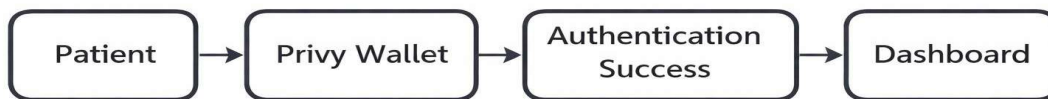


Figure .8

This interface ensures secure login using decentralized wallet authentication.

7.3.2 File Upload Interface :-

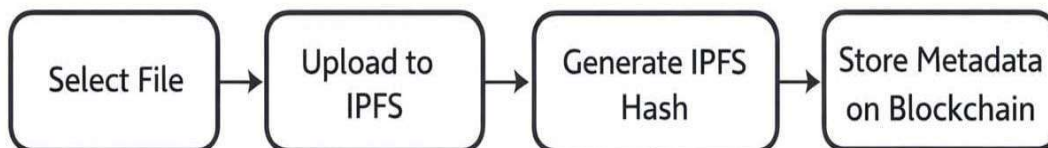


Figure 9

7.3.3 Record Viewing Interface :-

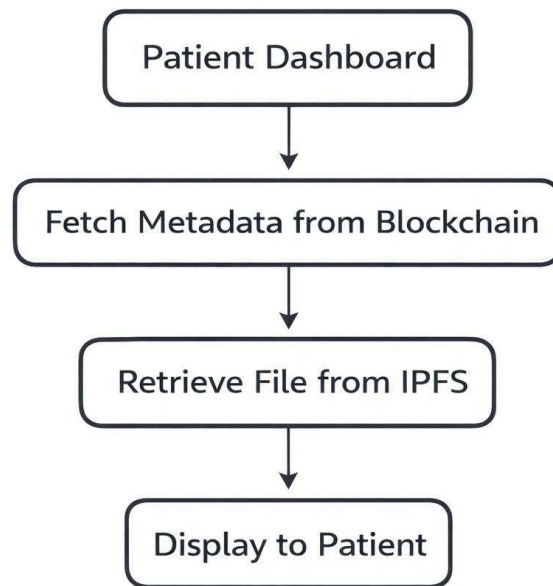


Figure. 10

CHAPTER8 PROJECT IMPLEMENTATION

8.1 Smart Contract Implementation Explanation :-

The smart contract in the VitalX system is implemented using Solidity and deployed on the Ethereum blockchain. The primary purpose of the smart contract is to store and manage metadata of medical records in an immutable and secure manner.

Each patient is uniquely identified by their wallet address, which acts as their digital identity. When a patient uploads a medical record, the actual file is stored on IPFS, and the returned IPFS hash along with the file name and timestamp is sent to the smart contract.

The smart contract performs the following functions:

- Accepts IPFS hash and file metadata from the frontend.
- Stores the metadata against the patient's wallet address.
- Ensures immutability of stored records.
- Allows retrieval of stored record metadata for viewing purposes.

By storing only metadata on-chain and keeping files off-chain, the system reduces gas costs while maintaining data integrity and security.

8.2 Frontend Overview :-

The frontend of VitalX is developed using React.js, providing a responsive and user-friendly interface. It allows patients to interact with the decentralized system without requiring deep blockchain knowledge.

Key Frontend Components :-

- **Landing Page:** Provides an overview of the system and navigation.
- **Login Page:** Enables secure wallet-based authentication using Privy.
- **Patient Dashboard:** Allows patients to upload medical records and view previously uploaded records.
- **Upload Module:** Handles file selection and initiates IPFS upload.
- **Records View Module:** Displays stored records with timestamps and access details.

8.3 Backend Overview :-

VitalX follows a decentralized backend architecture instead of a traditional centralized server-based backend.

Backend Components

- **Blockchain Network:** Acts as the backend for storing immutable metadata through smart contracts.
- **Smart Contracts:** Serve as the core logic layer for record storage and retrieval.
- **IPFS Network:** Functions as the decentralized storage backend for medical files.
- **Privy Authentication Service:** Manages wallet-based user authentication securely.

There is no centralized database in the system, which eliminates single points of failure and improves security and reliability.

8.4 Architecture Flow :-

The architecture flow explains how data moves through the system from user interaction to storage and retrieval.

Architecture Flow Steps :-

1. The patient accesses the VitalX web application.
2. The patient logs in using wallet-based authentication.
3. After successful authentication, the patient uploads a medical record.
4. The file is uploaded to IPFS, which generates a unique IPFS hash.
5. The IPFS hash and file metadata are sent to the smart contract.
6. The smart contract stores the metadata immutably on the blockchain.
7. For viewing records, metadata is fetched from the blockchain.
8. The file is retrieved from IPFS using the stored hash and displayed to the patient.

Architecture Flow Diagram (Textual Representation)

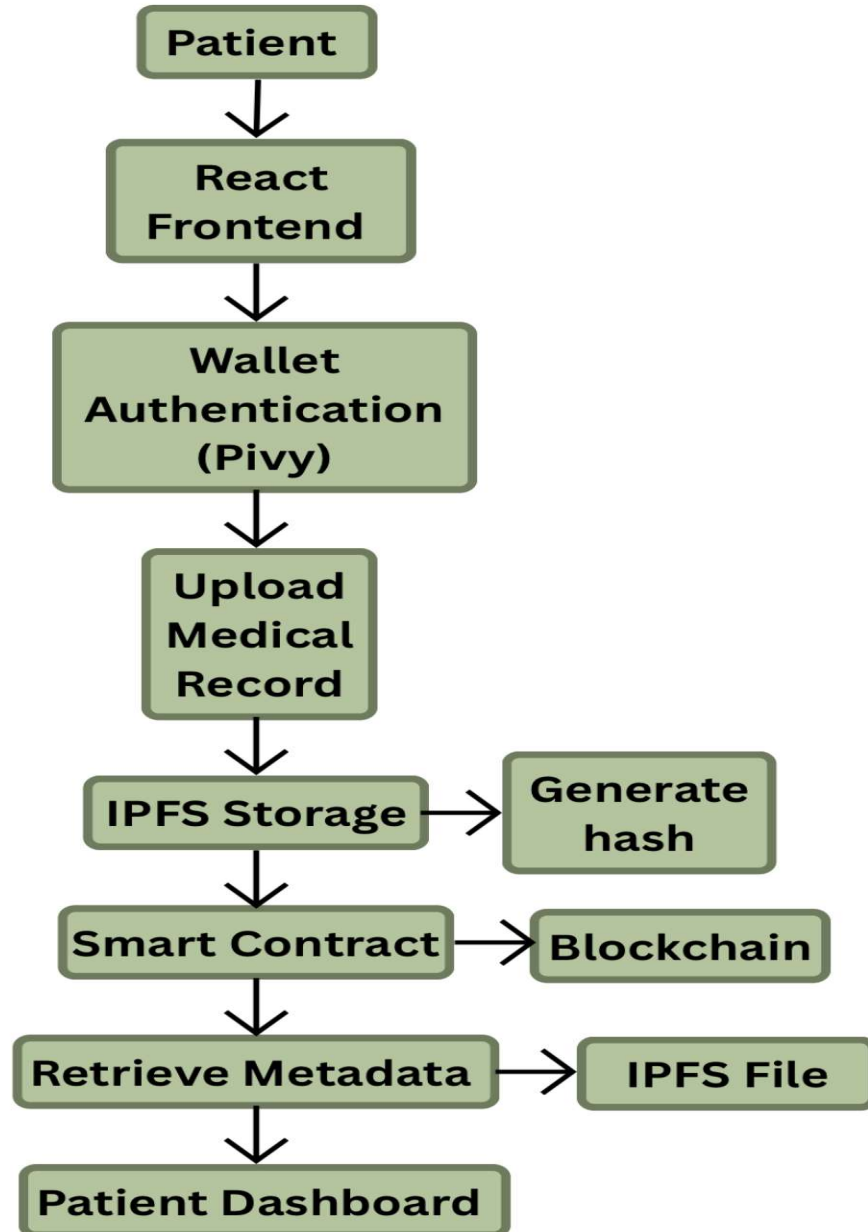


Figure. 11

CHAPTER 9:- TESTING PAGES

9.1 SYSTEM TESTING:-

System testing was performed to verify that the VitalX Ethereum-based application works correctly as an integrated system. The testing process ensured proper interaction between the frontend interface, blockchain smart contracts, wallet authentication, and IPFS-based decentralized storage.

The system was tested using a local Ethereum blockchain provided by Hardhat and MetaMask wallet integration.

9.2 Types of Testing Performed:-

9.2.1 Unit Testing :-

Unit testing was performed on individual components such as smart contract functions and frontend components to verify that each module works correctly in isolation.

9.2.2 Integration Testing :-

Integration testing ensured smooth interaction between:

- React frontend
- Privy/MetaMask wallet
- Ethereum smart contracts
- IPFS (Pinata) storage

This confirmed end-to-end functionality of the system.

9.2.3 Functional Testing :-

Functional testing validated core features such as:

- User login and signup
- Uploading medical records
- Storing metadata on blockchain
- Viewing uploaded records

9.2.4 Security Testing :-

Security testing focused on:

- Wallet-based authentication
- Ensuring only authorized users can upload or view records
- Protection against unauthorized blockchain access

9.2.5 Performance Testing :-

Performance testing was conducted to analyze:

- IPFS upload and retrieval time
- Blockchain transaction confirmation time

9.2.6 User Acceptance Testing :-

User acceptance testing ensured that the dashboard and user interface were easy to understand and usable by end users such as patients.

9.2.7 Test Case Tables :-

FIG 9.2.7 Automated Test Results got VitalXrecordStorage Smart Contract

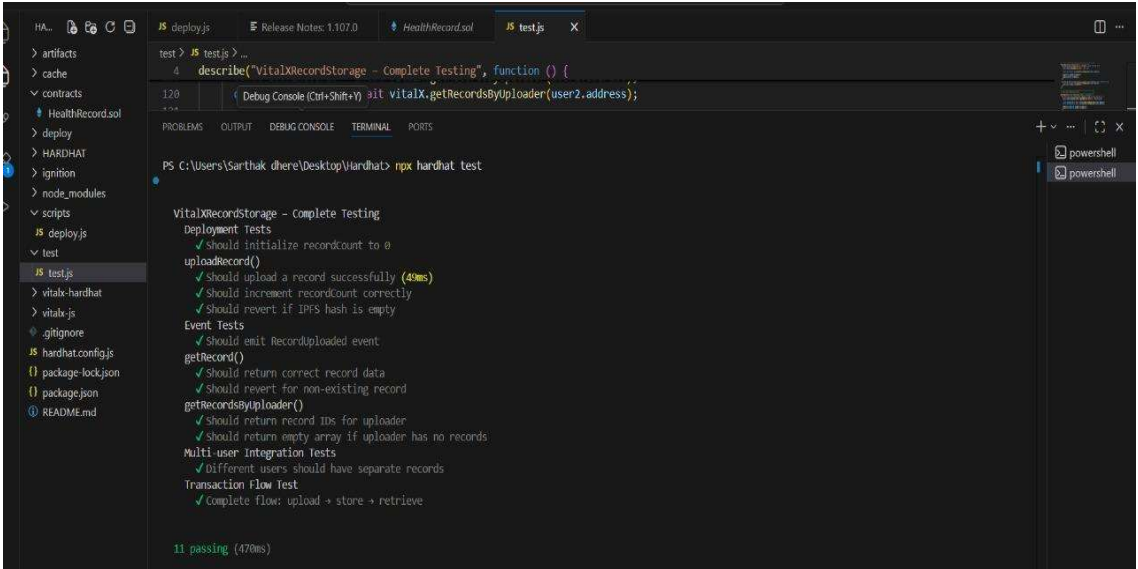


Figure. 12

Tes t Id	Test Description	Input Data	Expected Output	Actual Output	Status
1	Smart Contract Compilation	Solidity contract file	Contract compiles without errors	Contract compiled successfully	Pass
2	Smart Contract Deployment	Deployment script	Contract deployed on local blockchain	Contract deployed successfully	Pass
3	Store Medical Record Data	Record details	Data stored on blockchain	Data stored on blockchain	Pass
4	Retrieve Medical Record	Record ID	Correct record returned	Correct record returned	Pass
5	Unauthorized Access Validation	Invalid wallet address	Access denied	Access denied	Pass

Table. 6

CHAPTER 10 :-

SCREENSHOTS OF PROJECTS

10.1 Deployment :-

FIG 10.1 Smart Contract Deployment Transaction on Sepolia network.

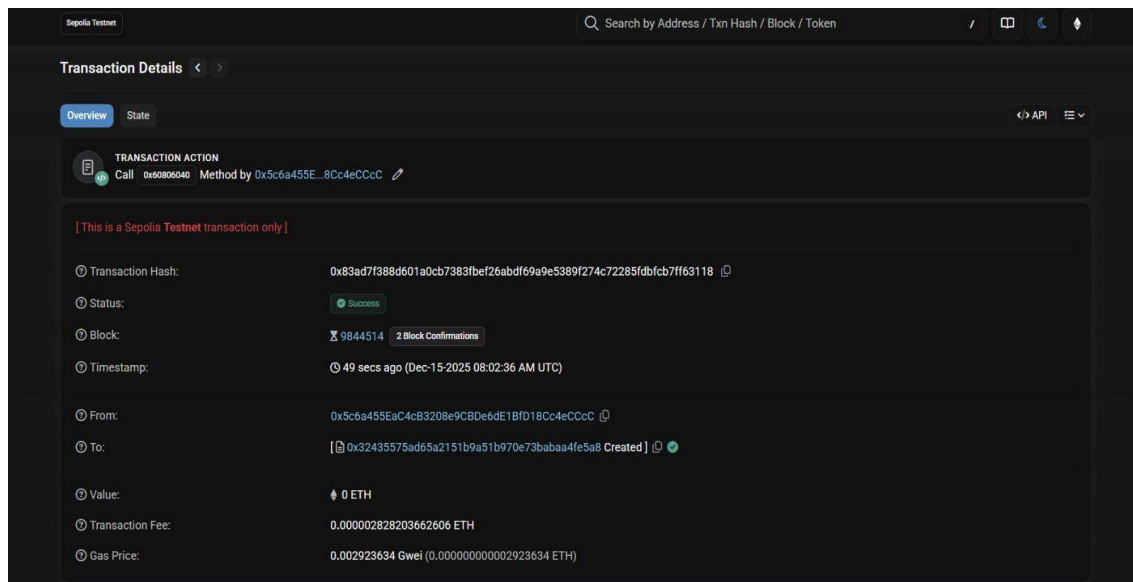


Figure. 13

10.2 Metamask Connection :-

FIG 10.2.Metamask connection process

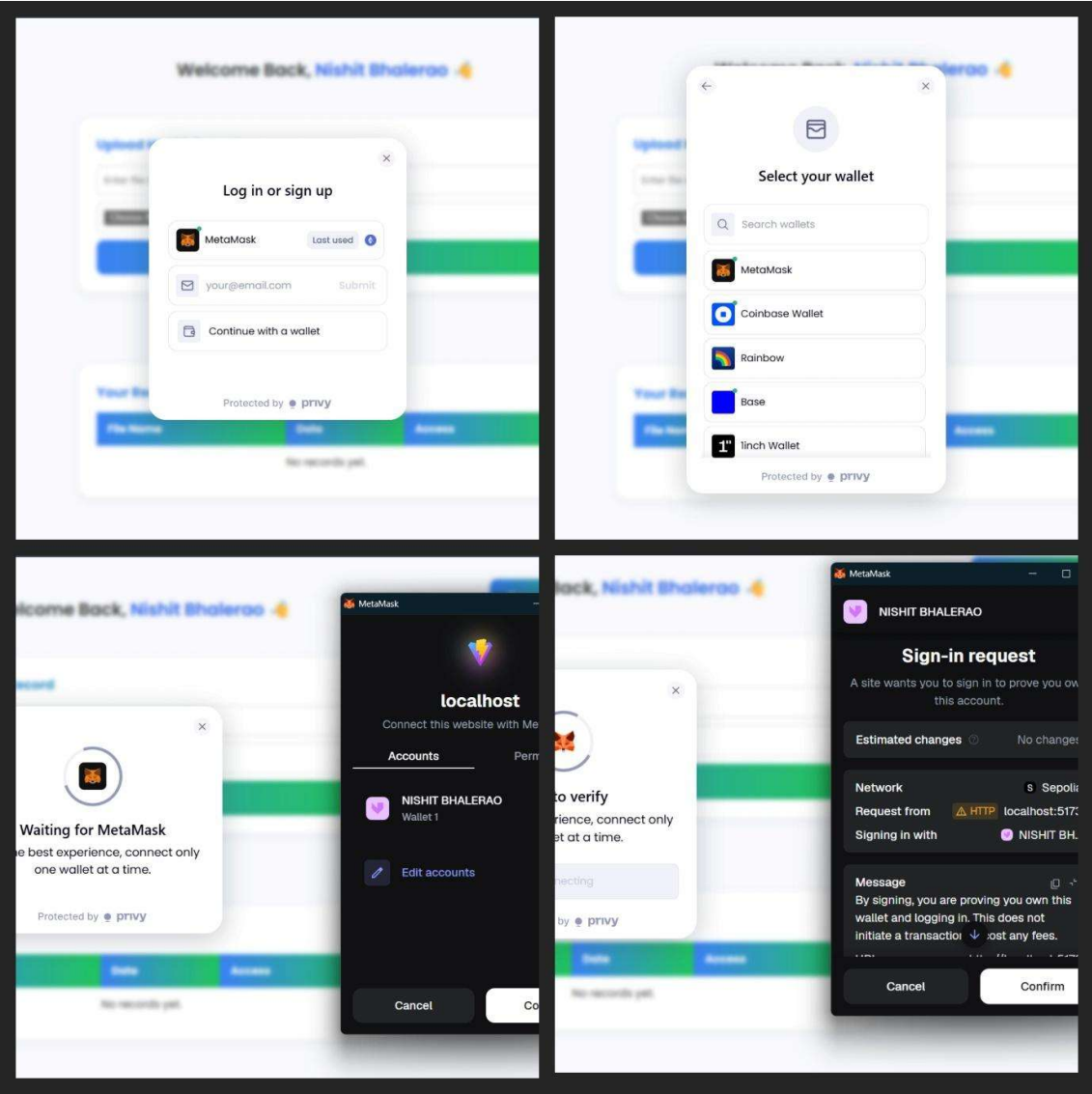


Figure. 14

10.3 UI Screens :-

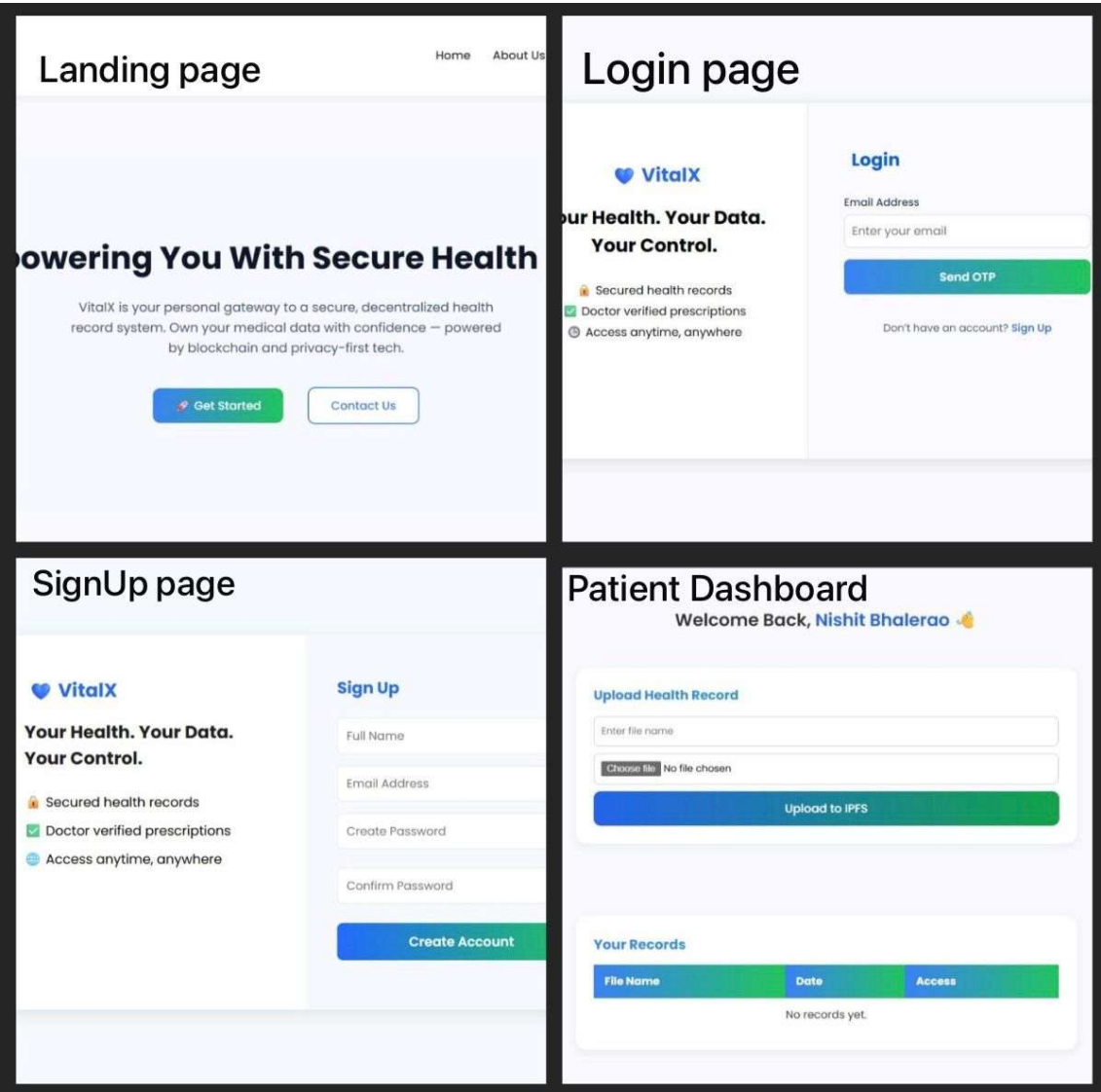


Figure. 15

10.4 User Flow :-

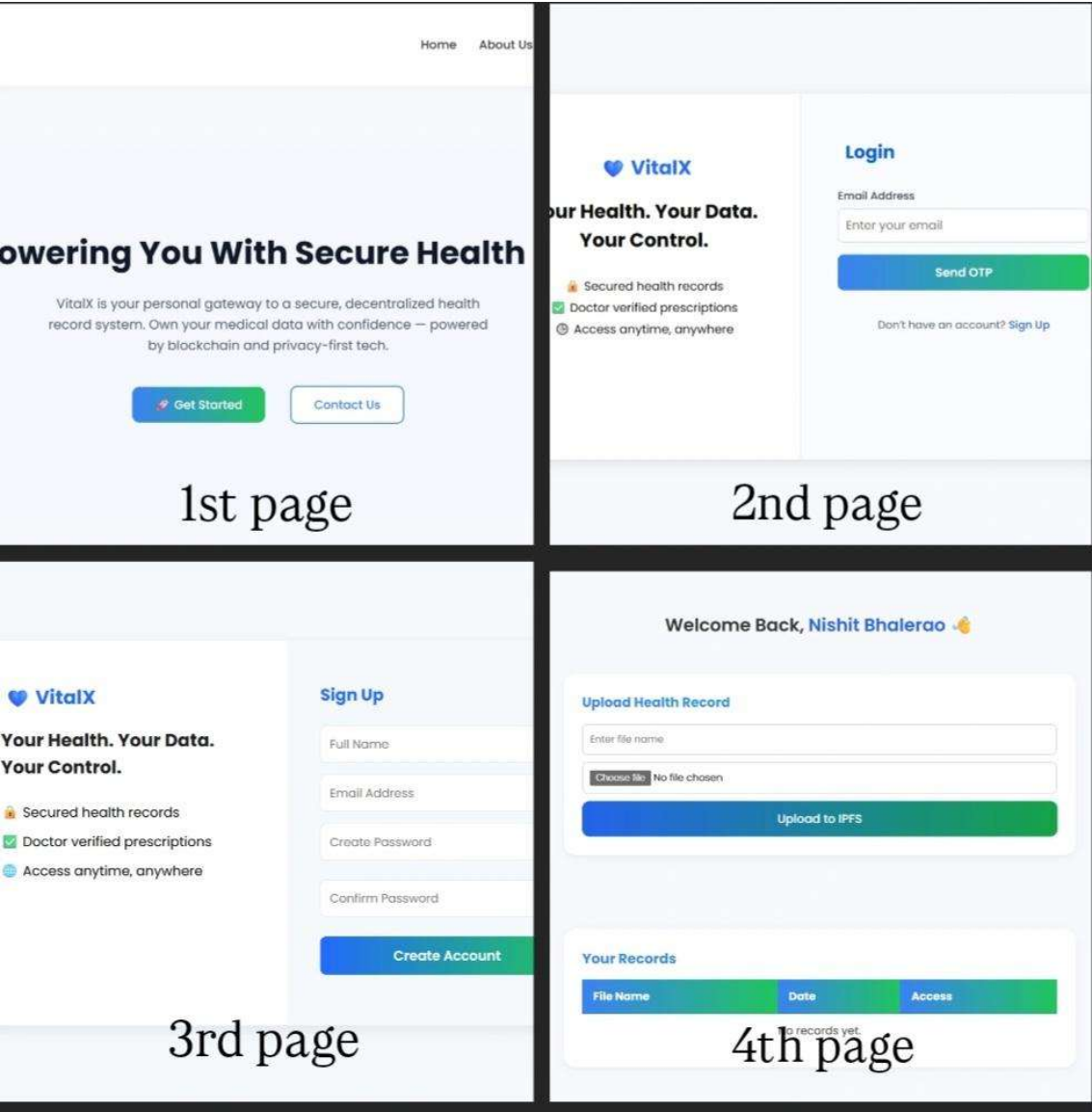


Figure .16

10.5 Contract Interactions :-

Record storing process

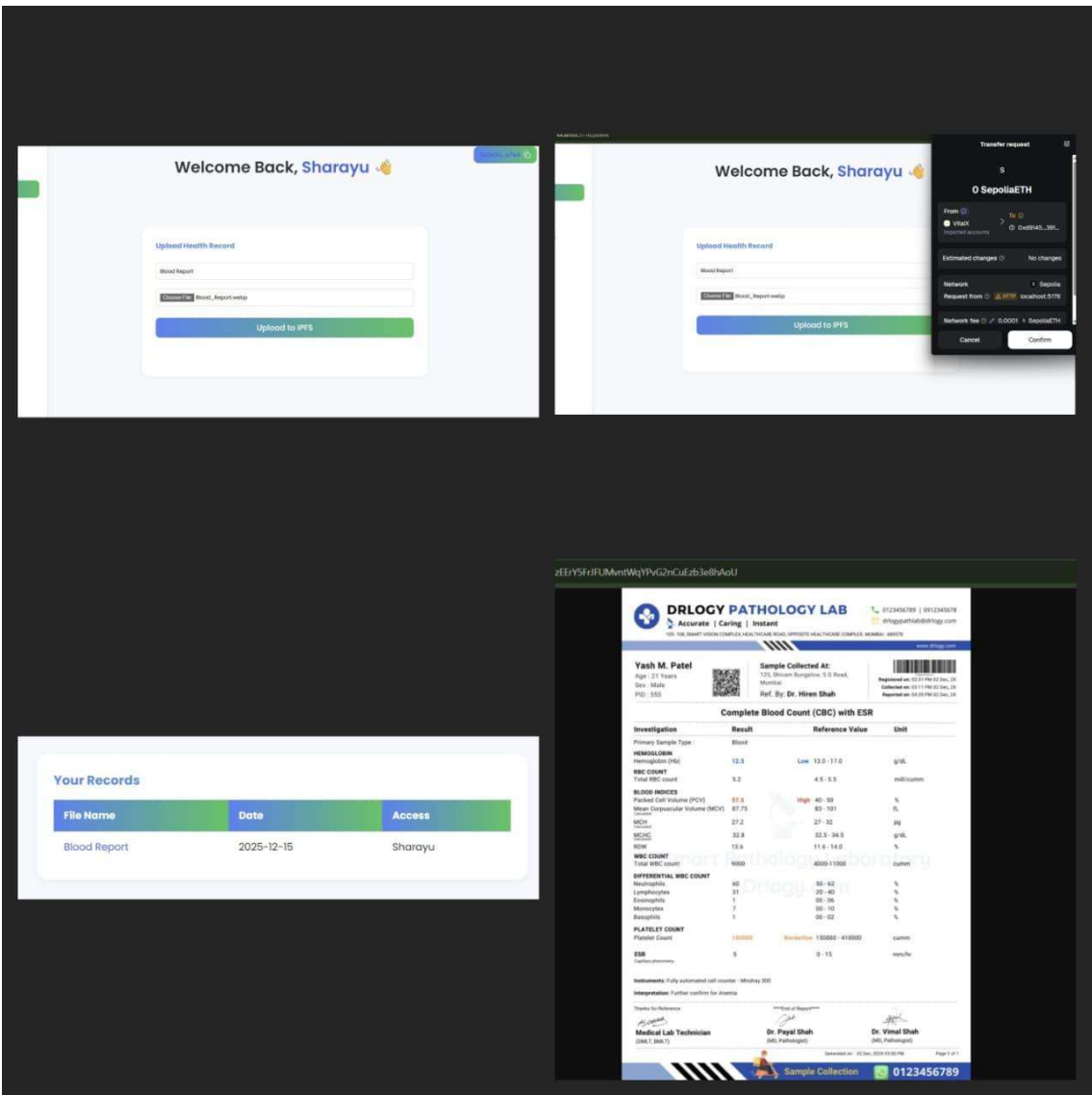


Figure .17

CHAPTER 11:-CODE SNIPPETS PAGES

11.1 Mapping & Events :-

```
// Auto-incrementing ID for records
uint256 public recordCount;

// ID => Record
mapping(uint256 => Record) public records;

// Uploader => list of their record IDs
mapping(address => uint256[]) private recordsByUploader;
```

Figure .18

11.2 Contract Declaration & Solidity Version:-

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract VitalXRecordStorage {
```

Figure .19

11.3 Record Structure :-

```
struct Record {
    uint256 id;           // Unique record ID
    address uploader;     // Address that uploaded the record
    string ipfsHash;      // IPFS CID of the file
    string label;         // Short description or filename
    uint256 timestamp;    // Upload time
}
```

Figure. 20

11.4 Record Upload Event :-

```
event RecordUploaded(  
    uint256 indexed id,  
    address indexed uploader,  
    string ipfsHash,  
    string label,  
    uint256 timestamp  
);
```

Figure. 21

CHAPTER 12 :- PROJECT TIMELINE

The development of the VitalX – Decentralized Health Record Management System was carried out in a structured and phased manner to ensure systematic progress, proper integration of technologies, and successful achievement of project objectives. The project was completed over multiple stages, starting from requirement analysis to final testing and documentation.

Phase 1: Requirement Analysis and Planning :-

- Studied existing centralized healthcare record systems and identified their limitations.
- Defined the problem statement, objectives, scope, and constraints of the project.
- Selected suitable technologies such as React.js, Blockchain, IPFS, Privy authentication, and Smart Contracts.
- Prepared initial system design and architecture.

Phase 2: System Design :-

- Designed the overall system architecture including frontend, blockchain layer, and IPFS storage.
- Created flow diagrams, use case diagrams, and data flow representations.
- Planned wallet-based authentication flow and patient dashboard structure.
- Designed smart contract logic for storing immutable record metadata.

Phase 3: Smart Contract Development :-

- Developed smart contracts using Solidity to store medical record metadata.
- Implemented functions for record creation and retrieval.
- Tested smart contracts for correctness, immutability, and security.
- Optimized contract logic to reduce blockchain gas costs

Phase 4: Frontend Development :-

- Developed the user interface using React.js and JSX.
- Implemented core pages such as Landing Page, Login Page, Signup Page, and Patient Dashboard.
- Integrated Privy wallet authentication for secure user login.
- Created file upload and record viewing components.

Phase 5: IPFS and Blockchain Integration :-

Integrated IPFS (via Pinata) for decentralized file storage.

- Connected frontend components with blockchain using Ethers.js.
- Implemented functionality to store IPFS hash and metadata on blockchain.
- Verified end-to-end flow from file upload to record retrieval.

Phase 6: Testing and Debugging :-

- Tested wallet authentication and access control.
- Verified file upload and retrieval from IPFS.
- Checked blockchain interactions and metadata storage.
- Fixed UI issues, routing errors, and integration bugs.

Phase 7: Documentation and Final Review :-

- Prepared project documentation including SRS, design, and implementation details.
- Reviewed system performance, security, and usability.
- Finalized project report and presentation.

CHAPTER 13 :-CHALLENGES AND SOLUTIONS

During the development of the VitalX system, several technical and implementation challenges were encountered. These challenges were addressed through careful analysis, experimentation, and optimization.

13.1 Issues Faced:-

13.1.1. Wallet-Based Authentication Integration :-

Integrating Web3 wallet authentication using Privy was initially challenging due to unfamiliar authentication flows and session handling in a React environment.

13.1. 2. IPFS File Upload and Retrieval :-

Ensuring reliable upload of medical records to IPFS and correctly retrieving them using the generated hash required proper handling of asynchronous operations and API responses.

13.1. 3. Blockchain Gas Cost Optimization :-

Storing data directly on the blockchain was expensive and inefficient, especially for large medical files.

13.1.4. Smart Contract–Frontend Communication :-

Connecting React frontend components with smart contracts using Ethers.js caused issues related to network configuration, contract ABI handling, and asynchronous data fetching.

13.1.5. UI and Routing Issues :-

Managing navigation between multiple pages such as login, signup, landing page, and dashboard led to routing conflicts and component rendering issues.

13.2 Solutions Implemented:-

13.2.1. Secure Wallet Authentication :-

The Privy authentication documentation was carefully studied and implemented correctly. Authentication state management was improved to ensure secure and consistent login sessions.

13.2.2. Reliable IPFS Integration :-

Pinata was used as an IPFS pinning service to ensure file availability. Proper error handling and async/await mechanisms were implemented to manage file uploads and retrieval.

13.2.3. Off-Chain File Storage Strategy :-

To reduce gas costs, only IPFS hash and metadata were stored on the blockchain, while actual medical files were stored off-chain on IPFS. This ensured efficiency and scalability.

13.2.4. Improved Blockchain Interaction :-

Ethers.js was used for smooth communication between frontend and blockchain. Contract ABIs were verified, and network configurations were properly handled.

13.2.5. Component-Based UI Structure

React components were modularized, and routing logic was improved to ensure smooth navigation across different pages.

13.2.6. Access Control via Wallet Address

Each medical record was linked to the patient's wallet address. Smart contracts ensured that records could only be accessed by the owner, maintaining privacy and security.

CHAPTER 14 :- RISKS & MITIGATION

This section identifies potential risks associated with the VitalX blockchain-based record storage system and explains the mitigation strategies adopted to minimize their impact. It also discusses security considerations and edge-case handling to ensure system reliability.

14.1 Project Risks :-

1.Smart contract bugs

Risk:

Errors in smart contract logic may lead to incorrect data storage, unexpected behavior, or irreversible loss of references.

14.1.1 Mitigation:

- Smart contracts were written using Solidity version ^0.8.20, which provides built-in overflow and underflow protection.
- Comprehensive testing was performed using Hardhat, covering deployment, record creation, retrieval, and failure scenarios.
- Modular and well-commented code was used to reduce complexity and improve readability.

14.1.2.Data Immutability Risk

Risk:

Once deployed, blockchain data cannot be modified or deleted, which may cause issues if incorrect data is stored.

Mitigation:

- Only IPFS hashes (file references) are stored on-chain instead of raw data.
- Validation checks ensure that empty or invalid IPFS hashes are rejected.
- Off-chain encryption of files ensures that even immutable data remains secure.

14.1.3. Dependency on Off-Chain Storage (IPFS)

Risk:

If IPFS nodes become unavailable, users may temporarily lose access to stored files.

Mitigation:

- Files are uploaded to IPFS before recording their hash on the blockchain.
- Pinning services can be used to ensure persistent availability of IPFS data.
- The blockchain always retains the reference, allowing re-pinning if needed.

14.1.4. Network and Environment Risks

Risk:

Deployment and interaction failures due to incorrect network configuration or unavailable blockchain nodes.

Mitigation:

- Hardhat local blockchain was used for development and testing to ensure a controlled environment.
- Deployment scripts were tested multiple times to validate stability.
- Clear deployment procedures were documented to reduce configuration errors.

14.2 Security Consideration :-

1. Unauthorized Access

Risk:

Unauthorized users may attempt to read or manipulate sensitive records.

Mitigation:

- Each record is associated with the uploader's Ethereum address.
- Access control logic can be extended to restrict record visibility to authorized roles (e.g., doctors or patients).
- Sensitive data is never stored directly on-chain.

2. Data Privacy

Risk:

Public blockchain visibility may expose sensitive information.

Mitigation:

- **Only encrypted IPFS hashes are stored on-chain.**
- **Actual medical or personal data is kept off-chain.**
- **Encryption ensures data confidentiality even if IPFS content is accessed.**

3.Transaction Integrity

Risk:

Malicious or malformed transactions may disrupt system functionality.

Mitigation:

- **Solidity require() statements validate inputs before execution.**
- **Invalid transactions are reverted, preserving blockchain state.**
- **Event logging provides transparency and traceability.**

14.3. Edge-Case Handling :-

1. Empty or Invalid Input

Case:-

User attempts to upload a record without an IPFS hash.

Handling:

- **The transaction is reverted with a clear error message.**

2. Non-Existent Record Access

Case:-

User requests a record ID that does not exist.

Handling:

- **The contract checks record boundaries and reverts the transaction.**

3 .Multiple Users Uploading Records

Case:-

Multiple users upload records simultaneously.

Handling:

- **Auto-incrementing record IDs ensure uniqueness.**
- **Separate mappings maintain record ownership per uploader.**

4. Empty Record Lists

Case:-

User has not uploaded any records.

Handling:

- **The system safely returns an empty array without errors.**

CHAPTER 15 :- GLOSSARY

15.1 Definitions of technical terms used :-

1.Blockchain:-

A decentralized and distributed digital ledger that records transactions securely and immutably across multiple nodes.

2.SmartContract:-

A self-executing program stored on a blockchain that automatically enforces rules and agreements without intermediaries.

3.Decentralization:-

The process of distributing control and data across a network rather than relying on a single central authority.

4.IPFS (InterPlanetary File System):-

A peer-to-peer distributed file storage system used to store large files in a decentralized manner.

5. IPFS Hash (CID):-

A unique cryptographic identifier generated by IPFS that represents the content of a stored file.

6. Web3:-

The decentralized version of the internet that uses blockchain, smart contracts, and wallets.

7. Wallet Authentication:-

A method of user authentication using cryptographic wallet addresses instead of usernames and passwords.

8. Privy Authentication:-

A Web3 authentication service used to securely manage wallet-based login and user identities.

9. React.js:-

A JavaScript library used for building interactive and component-based user interfaces.

10. Ethers.js:-

A JavaScript library used to interact with Ethereum blockchain and smart contracts.

11. Ethereum:-

A blockchain platform that supports smart contracts and decentralized applications.

12. Cryptographic Hash:-

A fixed-length digital fingerprint generated from data, used to verify integrity and authenticity.

13. Immutability:-

A property of blockchain data that prevents modification or deletion once recorded.

14. Metadata:-

Data that provides information about other data, such as file name, timestamp, and IPFS hash.

15. Decentralized Application (DApp):-

An application that runs on a blockchain network instead of a centralized server.

16. Gas Fee:-

The transaction fee required to execute operations on the Ethereum blockchain.

17. Node:-

A computer that participates in maintaining and validating the blockchain network.

18. IPFS Pinning:-

The process of keeping files permanently available on IPFS using pinning services.

19. Frontend:-

The user interface part of an application that users interact with directly.

20. Backend:-

The underlying logic and infrastructure that handle data processing and storage.

CHAPTER 16 :- CONCLUSION PAGE

16.1 What Was Achieved :-

The project VitalX – Decentralized Health Record Management System successfully demonstrates the use of blockchain technology to build a secure, decentralized, and patient-centric healthcare data management system. The system enables patients to upload and manage their medical records while maintaining full ownership and control over their data.

By integrating IPFS for decentralized file storage and smart contracts for immutable metadata logging, the project ensures data integrity, transparency, and security. The use of wallet-based authentication (Privy) eliminates the need for traditional usernames and passwords, thereby enhancing security and reducing dependency on centralized identity providers. Overall, the project achieves its objective of providing a reliable and tamper-proof solution for managing health records in a decentralized environment.

16.2 Improvements Possible :-

Although the system functions effectively, certain improvements can enhance its performance and usability:

1. Implementation of file encryption before storing data on IPFS to provide additional privacy.
2. Optimization of smart contracts to further reduce gas costs.
3. Improvement in user interface design for better accessibility and usability.
4. Integration of real-time notifications for record uploads and updates.
5. Performance optimization for handling a large number of records.

16.3 Future Scope :-

The VitalX system can be further extended with the following features in the future:

- 1. Introduction of doctor and hospital modules with controlled access permissions.**
- 2. Implementation of role-based access control using smart contracts.**
- 3. Integration with government health systems and insurance providers.**
- 4. Support for multi-chain deployment to reduce transaction costs.**
- 5. Use of advanced encryption techniques and zero-knowledge proofs for enhanced privacy.**
- 6. Development of a mobile application for wider accessibility.**

CHAPTER 17 :- REFERENCES

17.1 Websites :-

1. Ethereum Official Website – <https://ethereum.org>
2. InterPlanetary File System (IPFS) – <https://ipfs.tech>
3. React Official Website – <https://react.dev>
4. Privy Authentication – <https://www.privy.io>
5. Pinata IPFS Service – <https://www.pinata.cloud>
6. Supabase Official Website – <https://supabase.com>

17.2 Tutorials :-

1. React Documentation Tutorials – <https://react.dev/learn>
2. Solidity Beginner Tutorials – <https://docs.soliditylang.org>
3. Ethereum DApp Development Guide – <https://ethereum.org/developers>
4. IPFS Upload Tutorials – <https://docs.ipfs.tech/how-to/store-data>
5. Web3 Authentication Tutorials – <https://www.privy.io/docs>

17.3 IEEE Papers :-

1. S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” IEEE, 2008.
2. M. Crosby et al., “Blockchain Technology: Beyond Bitcoin,” *IEEE Computer Society*, 2016.
3. K. Christidis and M. Devetsikiotis, “Blockchains and Smart Contracts for the Internet of Things,” *IEEE Access*, 2016.
4. A. Azaria et al., “MedRec: Using Blockchain for Medical Data Access and Permission Management,” *IEEE Open & Big Data Conference*, 2016.
5. Y. Zhanget al., “Healthcare Data Sharing Using Blockchain Technology,” *IEEE Access*, 2018.

17.4 Documentation :-

1. Ethereum Developer Documentation – <https://ethereum.org/developers/docs>
2. Solidity Language Documentation – <https://docs.soliditylang.org>
3. IPFS Technical Documentation – <https://docs.ipfs.tech>
4. React.js API Documentation – <https://react.dev/reference>
5. Privy Developer Documentation – <https://www.privy.io/docs>
6. Supabase Documentation – <https://supabase.com/docs>